Nesne Temelli Programlama (OOP) Araştırma Ödevi

Giriş

Bu çalışmada nesne yönelimli programlamanın (OOP) temel kavramlarını araştırdım. Daha önce sadece fonksiyonlarla kod yazıyordum, her şeyi düz bir şekilde alt alta yazıyordum açıkçası. OOP'yi araştırdıkça aslında büyük projelerde neden bu kadar önemli olduğunu anlamaya başladım. Aşağıda her kavramı elimden geldiğince açıklamaya ve örneklendirmeye çalıştım.

1. Sınıf (Class) ve Nesne (Object)

Sınıf, bir şeyin şablonu, kalıbı gibi düşünülebilir. Mesela bir araba fabrikasındaki teknik çizim gibi. Çizimin kendisi araba değil ama o çizimden arabalar üretiliyor. İşte sınıf da böyle bir şey.

Nesne ise o şablondan üretilen gerçek şey. Yani fabrikanın ürettiği her bir araba birer nesne oluyor.

Günlük hayattan bakarsak: Bir kurabiye kalıbımız var diyelim. Kalıbın kendisi sınıf, o kalıpla kestiğimiz her kurabiye de bir nesne.

```
class Araba
{
    public string Marka;
    public string Renk;
    public int Yil;

    public void Calistir()
    {
        Console.WriteLine(Marka + " çalıştırıldı.");
    }
}

// nesne olusturma
Araba araba1 = new Araba();
araba1.Marka = "Toyota";
araba1.Renk = "Beyaz";
araba1.Yil = 2020;
araba1.Calistir();  // Toyota çalıştırıldı.

Araba araba2 = new Araba();
araba2.Marka = "Honda";
araba2.Renk = "Siyah";
araba2.Yil = 2022;
```

Burada `Araba` sınıfından iki tane farklı nesne oluşturduk. İkisi de araba ama farklı özelliklere sahipler. Bence bu şablondan üretme mantığı OOP'nin en temel fikri.

---

2. Encapsulation (Kapsülleme)

Kapsülleme, bir nesnenin iç detaylarını dışarıdan gizleme işi. Yani biz bir nesneyi kullanırken onun içinde neler döndüğünü bilmemize gerek yok, sadece bize sunulan arayüzü kullanıyoruz.

Günlük hayattan örnek verecek olursam: Televizyonun kumandasında ses açma tuşuna basıyoruz, ses açılıyor. İçeride hangi devre çalışıyor, hangi sinyal gönderiliyor bilmiyoruz ve bilmemize de gerek yok.

```
class BankaHesabi
{
    private double bakiye;  // disardan erisilemez

    public BankaHesabi(double ilkBakiye)
    {
        bakiye = ilkBakiye;
    }

    public void ParaYatir(double miktar)
    {
        if (miktar > 0)
        {
            bakiye += miktar;
            Console.WriteLine(miktar + " TL yatırıldı.");
        }
        else
        {
            Console.WriteLine("Geçersiz miktar!");
        }
    }

    public void ParaCek(double miktar)
    {
        if (miktar > 0 && miktar <= bakiye)
        {
            bakiye -= miktar;
            Console.WriteLine(miktar + " TL çekildi.");
        }
        else
        {
            Console.WriteLine("Yetersiz bakiye veya geçersiz miktar.");
        }
    }

    public double BakiyeGor()
    {
        return bakiye;
    }
}
```

Burada `bakiye` değişkeni private yani dışardan kimse doğrudan `hesap.bakiye = -500` gibi saçma bir değer atayamaz. Para yatırma ve çekme işlemleri metotlar üzerinden kontrollü bir şekilde yapılıyor. Bu sayede veri bütünlüğü korunmuş oluyor.

Kapsüllemenin asıl çözdüğü problem şu: eğer herkes her değişkene istediği gibi erişebilseydi, büyük projelerde bir yerden bir değeri yanlışlıkla değiştirip bulmakta zorlanırdık. Kapsülleme bunu engelliyor.

3. Abstraction (Soyutlama)

Soyutlama, karmaşık sistemlerin sadece gerekli kısımlarını gösterip gereksiz detayları gizleme. Kapsülleme ile biraz karışıyor ama aralarında fark var. Kapsülleme veriyi gizlerken, soyutlama daha çok karmaşıklığı gizliyor.

Mesela biz araba kullanırken direksiyonu çeviriyoruz, gaza basıyoruz. Ama arkada hidrolik sistemin nasıl çalıştığını, motorun hangi silindirine yakıt gittiğini bilmiyoruz. Araba bize soyutlanmış bir arayüz sunuyor: direksiyon, gaz, fren.

```
abstract class Sekil
{
    public abstract double AlanHesapla();  // nasıl hesaplanacağı alt sınıfa bırakılmış

    public void Bilgi()
    {
        Console.WriteLine("Bu bir geometrik şekildir.");
    }
}

class Dikdortgen : Sekil
{
    public double En;
    public double Boy;

    public override double AlanHesapla()
    {
        return En * Boy;
    }
}

class Daire : Sekil
{
    public double YariCap;

    public override double AlanHesapla()
    {
        return 3.14 * YariCap * YariCap;
    }
}
```

Burada `Sekil` sınıfı soyut. "Alan hesapla" diyor ama nasıl hesaplanacağını bilmiyor çünkü bu şekle göre değişir. Her alt sınıf kendi hesaplama yöntemini yazıyor. Bu şekilde bir şeklin ne olduğunu

bilmesek bile `AlanHesapla()` diyebiliyoruz.

---

4. Inheritance (Kalıtım)

Kalıtım, bir sınıfın başka bir sınıftan özellik ve davranışları miras alması. Bu sayede aynı kodu tekrar tekrar yazmaktan kurtuluyoruz.

Günlük hayattan örnek: Bir "Hayvan" düşünelim. Her hayvanın adı var, sesi var, yemek yeme davranışı var. Kedi de bir hayvan, köpek de. Ortak olan şeyleri "Hayvan" sınıfına yazıyoruz, farklı olan şeyleri de alt sınıflara.

```
class Hayvan
{
    public string Ad;
    public int Yas;

    public void YemekYe()
    {
        Console.WriteLine(Ad + " yemek yiyor.");
    }
}

class Kedi : Hayvan
{
    public void Miyavla()
    {
        Console.WriteLine(Ad + " miyavlıyor!");
    }
}

class Kopek : Hayvan
{
    public void Havla()
    {
        Console.WriteLine(Ad + " havlıyor!");
    }
}

// kullanim
Kedi kedi = new Kedi();
kedi.Ad = "Tekir";
kedi.Yas = 3;
kedi.YemekYe();   // Tekir yemek yiyor.
kedi.Miyavla();   // Tekir miyavlıyor!
```

`Kedi` sınıfında `YemekYe` metodunu yazmadık ama `Hayvan`'dan miras aldığı için kullanabiliyoruz. Kalıtımın en büyük avantajı kod tekrarını azaltması. Gerçek projelerde de mesela bir e-ticaret sitesinde `Urun` adında bir temel sınıf olur, ondan `Elektronik`, `Giyim`, `Yiyecek` gibi alt sınıflar türetilir.

---

5. Polymorphism (Çok Biçimlilik)

Polimorfizm, aynı metodun farklı sınıflarda farklı şekilde çalışması demek. Kelime anlamı "çok biçimlilik" zaten bunu söylüyor.

Günlük hayattaki karşılığı şu olabilir: "Ses çıkar" dediğimizde kedi miyavlar, köpek havlar, kuş cıvıldar. Hepsine aynı komutu veriyoruz ama her biri farklı davranıyor.

```
class Hayvan
{
    public string Ad;

    public virtual void SesCikar()
    {
        Console.WriteLine("Hayvan ses çıkarıyor.");
    }
}

class Kedi : Hayvan
{
    public override void SesCikar()
    {
        Console.WriteLine(Ad + ": Miyav!");
    }
}

class Kopek : Hayvan
{
    public override void SesCikar()
    {
        Console.WriteLine(Ad + ": Hav hav!");
    }
}

class Kus : Hayvan
{
    public override void SesCikar()
    {
        Console.WriteLine(Ad + ": Cik cik!");
    }
}

// polimorfizm ornegi
List<Hayvan> hayvanlar = new List<Hayvan>();
hayvanlar.Add(new Kedi { Ad = "Pamuk" });
hayvanlar.Add(new Kopek { Ad = "Karabas" });
hayvanlar.Add(new Kus { Ad = "Mavis" });

foreach (Hayvan h in hayvanlar)
{
    h.SesCikar();  // her biri kendi sesini cikarir
}
// Pamuk: Miyav!
// Karabas: Hav hav!
// Mavis: Cik cik!
```

Buradaki güzel nokta şu: `foreach` döngüsünde hepsine `Hayvan` gözüyle bakıyoruz ama her biri kendi `SesCikar` metodunu çalıştırıyor. Bu büyük projelerde çok işe yarıyor çünkü yeni bir hayvan tipi eklediğimizde döngüyü değiştirmemize gerek kalmıyor, sadece yeni sınıfı yazıp listeye ekliyoruz.

---

6. Constructor ve Nesne Oluşturma Mantığı

Constructor (yapıcı metot), bir nesne oluşturulurken otomatik çalışan özel metot. Nesnenin ilk değerlerini ayarlamak için kullanılıyor.

Bunu şöyle düşünebiliriz: Bir bebek doğduğunda hemen bir ismi oluyor, cinsiyeti belli. Bunlar doğuştan gelen özellikler. Constructor da nesnenin "doğum anı" gibi.

```
class Ogrenci
{
    public string Ad;
    public string Soyad;
    public int Numara;

    // parametresiz constructor
    public Ogrenci()
    {
        Ad = "Bilinmiyor";
        Soyad = "Bilinmiyor";
        Numara = 0;
    }

    // parametreli constructor
    public Ogrenci(string ad, string soyad, int numara)
    {
        Ad = ad;
        Soyad = soyad;
        Numara = numara;
    }

    public void BilgiGoster()
    {
        Console.WriteLine(Numara + " - " + Ad + " " + Soyad);
    }
}

// kullanim
Ogrenci ogr1 = new Ogrenci();  // parametresiz
Ogrenci ogr2 = new Ogrenci("Ali", "Yılmaz", 101);  // parametreli

ogr1.BilgiGoster();  // 0 - Bilinmiyor Bilinmiyor
ogr2.BilgiGoster();  // 101 - Ali Yılmaz
```

Constructor olmasaydı her nesne oluşturduktan sonra tek tek her özelliği atamamız gerekirdi ve belki birini unuturduk. Constructor bu ilk ayarlamaları zorunlu kılarak hataları önlüyor.

Bir sınıfta birden fazla constructor olabilir buna **constructor overloading** deniyor. Farklı parametrelerle farklı şekillerde nesne oluşturabiliyoruz.

---

7. Access Modifiers (Erişim Belirleyiciler)

Erişim belirleyiciler, bir sınıfın veya üyenin nereden erişilebileceğini kontrol eden anahtar kelimeler.

Modifier	Açıklama
public	Her yerden erişilebilir
private	Sadece kendi sınıfı içinden erişilebilir
protected	Kendi sınıfı ve ondan türeyen sınıflardan erişilebilir

Günlük hayat benzetmesi: Bir evde düşünelim. - **public**: Evin bahçesi, herkes görebilir. - **private**: Evin kasası, sadece ev sahibi açabilir. - **protected**: Evin anahtarı, sadece aile bireyleri (miras alan sınıflar) kullanabilir.

```
class Calisan
{
    public string Ad;           // herkes gorebilir
    private double maas;        // sadece bu sinif icinden
    protected string departman; // bu sinif ve alt siniflar

    public Calisan(string ad, double maas, string departman)
    {
        this.Ad = ad;
        this.maas = maas;
        this.departman = departman;
    }

    public double MaasGoster()
    {
        return maas;  // private alana kendi icinden erisiyoruz
    }
}

class Mudur : Calisan
{
    public Mudur(string ad, double maas, string dep) : base(ad, maas, dep)
    {
    }

    public void DepartmanGoster()
    {
        Console.WriteLine("Departman: " + departman);
        // departman protected oldugu icin alt siniftan erisilebilir

        // Console.WriteLine(maas);  // bu hata verir cunku maas private
    }
}
```

Access modifier'lar aslında kapsüllemenin uygulama aracı. Neyin nereden erişilebileceğini biz belirliyoruz. Bu büyük ekiplerde çok önemli çünkü bir geliştirici yanlışlıkla başka birinin sınıfındaki kritik bir değeri bozamıyor.

---

8. Interface ve Abstract Class

Abstract Class (Soyut Sınıf)

Abstract class, doğrudan nesne oluşturulamayan, alt sınıflara temel oluşturan sınıf. Hem normal metotları hem de gövdesiz (abstract) metotları barındırabilir.

```
abstract class Calisan
{
    public string Ad;

    public void IseCikar()
    {
        Console.WriteLine(Ad + " işe gidiyor.");  // ortak davranis
    }

    public abstract double MaasHesapla();  // her calisan tipi farkli hesaplar
}

class TamZamanli : Calisan
{
    public double AylikMaas;

    public override double MaasHesapla()
    {
        return AylikMaas;
    }
}

class YariZamanli : Calisan
{
    public double SaatUcreti;
    public int CalismaSaati;

    public override double MaasHesapla()
    {
        return SaatUcreti * CalismaSaati;
    }
}
```

Interface (Arayüz)

Interface ise bir sözleşme gibi. "Bu interface'i uygulayan sınıf şu metotlara sahip olmak zorundadır" diyor. İçinde sadece metot imzaları var, gövde yok (C#'ın yeni sürümlerinde default implementation eklenmiş ama temelde böyle).

```
interface IYazdirabilir
{
    void Yazdir();
}

interface IKaydedilebilir
{
    void Kaydet();
}

class Rapor : IYazdirabilir, IKaydedilebilir
{
    public string Baslik;
    public string Icerik;

    public void Yazdir()
    {
        Console.WriteLine("Rapor yazdırılıyor: " + Baslik);
    }

    public void Kaydet()
    {
        Console.WriteLine("Rapor kaydediliyor: " + Baslik);
    }
}
```

Aralarındaki Fark

Bunu araştırırken en çok karıştırdığım konu buydu. Farkları şöyle özetleyebilirim:

- Abstract class'tan sadece bir tane miras alınabilir ama birden fazla interface uygulanabilir.
- Abstract class'ta hem gövdeli hem gövdesiz metot olabilir, interface'te (genel olarak) sadece gövdesiz metot imzaları olur.
- Abstract class "bu nedir" sorusuna cevap verir (Kedi bir Hayvandır), interface ise "bu ne yapabilir" sorusuna cevap verir (Rapor yazdırılabilir, kaydedilebilir).

Bir sınıf hem abstract class'tan türeyip hem de interface uygulayabilir. Bu da esneklik sağlıyor.

---

9. Composition ve Inheritance Arasındaki Fark

Bu konu aslında "ne zaman kalıtım, ne zaman composition kullanmalıyız" sorusunun cevabı.

Inheritance (Kalıtım): "is-a" ilişkisi kurar. Kedi **bir** hayvandır.

Composition: "has-a" ilişkisi kurar. Araba **bir** motora **sahiptir**.

```
// Inheritance ornegi - "is-a"
class Hayvan
{
    public void Nefes()
    {
        Console.WriteLine("Nefes alıyor.");
    }
}

class Kedi : Hayvan  // Kedi bir Hayvandır
{
}

// Composition ornegi - "has-a"
class Motor
{
    public int BeygirGucu;

    public void Calis()
    {
        Console.WriteLine("Motor çalışıyor. Güç: " + BeygirGucu + " HP");
    }
}

class Araba
{
    private Motor motor;  // Araba bir Motor degil, motora sahip

    public Araba(int beygir)
    {
        motor = new Motor();
        motor.BeygirGucu = beygir;
    }

    public void Calistir()
    {
        motor.Calis();
        Console.WriteLine("Araba hareket ediyor.");
    }
}
```

Kalıtım kullanırken dikkat etmemiz gereken şey şu: gerçekten "bu onun bir türü mü" diye sormamız lazım. Eğer sadece bazı özelliklerini kullanmak istiyorsak composition daha mantıklı oluyor.

Araştırırken karşıma çıkan güzel bir kural: **"Favor composition over inheritance"** yani mümkünse kalıtım yerine composition tercih edilmeli. Çünkü kalıtım sıkı bir bağ kuruyor, üst sınıf değişirse alt sınıflar da etkileniyor. Composition'da bu bağ daha gevşek.

10. Nesneler Arasındaki İlişkiler

Nesneler arasında farklı tipte ilişkiler olabiliyor. Bunları araştırdığımda 4 ana tip gördüm:

Association (İlişkilendirme)

İki nesne birbirini tanıyor ama birbirinden bağımsız yaşayabiliyorlar. Mesela öğretmen ve öğrenci: öğretmen olmasa da öğrenci var, öğrenci olmasa da öğretmen var.

```
class Ogretmen
{
    public string Ad;
}

class Ogrenci
{
    public string Ad;
    public Ogretmen Danismani;  // ogrenci bir ogretmeni tanir ama sahiplenmez
}
```

Aggregation (Birleştirme)

"Has-a" ilişkisi ama parçalar bağımsız yaşayabilir. Bir takımın oyuncuları var ama takım dağılsa oyuncular yok olmaz.

```
class Oyuncu
{
    public string Ad;
}

class Takim
{
    public string TakimAdi;
    public List<Oyuncu> Oyuncular;  // takim oyunculara sahip ama onlari olusturmuyor

    public Takim(string ad)
    {
        TakimAdi = ad;
        Oyuncular = new List<Oyuncu>();
    }

    public void OyuncuEkle(Oyuncu oyuncu)
    {
        Oyuncular.Add(oyuncu);
    }
}
```

Composition (Bileşim)

"Has-a" ilişkisi ama parçalar bütün olmadan yaşayamaz. Bir evin odaları var, ev yıkılırsa odalar da yok olur.

```
class Oda
{
    public string OdaAdi;

    public Oda(string ad)
    {
        OdaAdi = ad;
    }
}

class Ev
{
    private List<Oda> odalar;

    public Ev()
    {
        // odalar Ev icinde olusturuluyor, Ev olmadan var olamazlar
        odalar = new List<Oda>();
        odalar.Add(new Oda("Salon"));
        odalar.Add(new Oda("Yatak Odası"));
        odalar.Add(new Oda("Mutfak"));
    }
}
```

Dependency (Bağımlılık)

Bir sınıf başka bir sınıfı geçici olarak kullanır, kalıcı bir sahiplik yok. Mesela bir yazıcı sınıfı, belge sınıfını parametre olarak alıp yazdırır.

```
class Belge
{
    public string Icerik;
}

class Yazici
{
    public void Yazdir(Belge belge)  // gecici kullanim, sahiplenmez
    {
        Console.WriteLine("Yazdırılıyor: " + belge.Icerik);
    }
}
```

Sonuç

OOP'yi araştırınca aslında günlük hayattaki birçok şeyin zaten nesne mantığıyla çalıştığını fark ettim. Bir arabayı kullanırken içindeki karmaşık mekaniği bilmemize gerek yok (soyutlama ve kapsülleme), farklı araç tipleri ortak özellikleri paylaşıyor (kalıtım), aynı komuta farklı tepki veriyorlar (polimorfizm).

Bu kavramları tek tek öğrenmek kolay gibi gözükse de asıl zorluk hangisini ne zaman kullanacağımızı bilmek. Özellikle composition vs inheritance kararı ve interface ile abstract class ayrımı pratikte deneyim gerektiren konular gibi duruyor. Bundan sonra yazdığım kodlarda bu prensipleri uygulamaya çalışacağım.

Kaynaklar:

1. Microsoft, "Object-Oriented Programming (C#)", Microsoft Learn, https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/ (Erişim Tarihi: 25.10.2026)
2. W3Schools, "C# OOP", https://www.w3schools.com/cs/cs_oop.php (Erişim Tarihi: 24.10.2026)
3. GeeksforGeeks, "C# | Object Oriented Programming", https://www.geeksforgeeks.org/c-sharp-object-oriented-programming/ (Erişim Tarihi: 26.10.2026)
4. Vikipedi, "Nesne Yönelimli Programlama", https://tr.wikipedia.org/wiki/Nesne_y%C3%B6nelimli_programlama (Erişim Tarihi: 24.10.2026)
5. BTK Akademi, "C# Programlama Dersleri", https://www.btkakademi.gov.tr/ (Erişim Tarihi: 27.10.2026)
6. Engin Demiroğ, "C# Eğitim Serisi", YouTube, https://www.youtube.com/@engindemirog (Erişim Tarihi: 26.10.2026)

GitHub Repo: https://github.com/hasanaliyucel-06/Web-Programciligi-lll